Project 1:
ILP Limit Study

Version 1

ECE 721: Advanced Microarchitecture
Spring 2026, Prof. Rotenberg

Due: Saturday, January 31, 2026, 11:59pm
Late projects will NOT be accepted because Project 1 is submitted as a Moodle quiz.

- READ THIS ENTIRE DOCUMENT.

1. Introduction

In this project, you will use 721sim (released only as a binary executable at this point) to study ILP bottlenecks in a subset of the SPEC benchmarks. You will explore the effect that the following features have on the IPC measured for a 16-wide superscalar processor:

- Sizes of ILP-extracting structures: Active List (AL), Load Queue/Store Queue (LQ/SQ), and Issue Queue (IQ). Note: by not explicitly specifying the size of the Physical Register File (PRF), 721sim defaults to auto-sizing the PRF conservatively (upper bound) with respect to the AL size: PRF size = AL size + # architectural registers.
- Data cache: perfect (always hit in L1 data cache) vs. real memory hierarchy (L1, L2, L3).
- Branch prediction: perfect vs. real.
- Trace cache: perfect trace cache vs. no trace cache.
- Memory dependence prediction: oracle vs. real (three different variants).

The following parameters are fixed for Project 1, by virtue of specifying the `-L` (limit study) command-line argument.

- The fetch, dispatch, issue, and retire widths are all 16. **Thus, the upper bound on IPC for this limit study is 16 instr./cycle.**
- Each of the 16 execution lanes can execute any instruction type (universal execution lanes) with 1-cycle EX latency.
- The IQ uses ideal age-based scheduling.
- There are 64 branch checkpoints (the maximum supported by 721sim). You will learn about this concept in upcoming lectures and implement this concept as part of Project 2.
- Fetch Queue (FQ) size is 64.

Note: After running the simulator, you can see all parameters of the modeled superscalar processor in the `stats.*` output file from 721sim.

Note: If you are curious, you can run "`./721sim -h`" to get a complete list of command-line arguments.

2. Stuff you need

2.1. 721sim executable

The 721sim executable is posted on the Moodle site.

2.2. Benchmarks

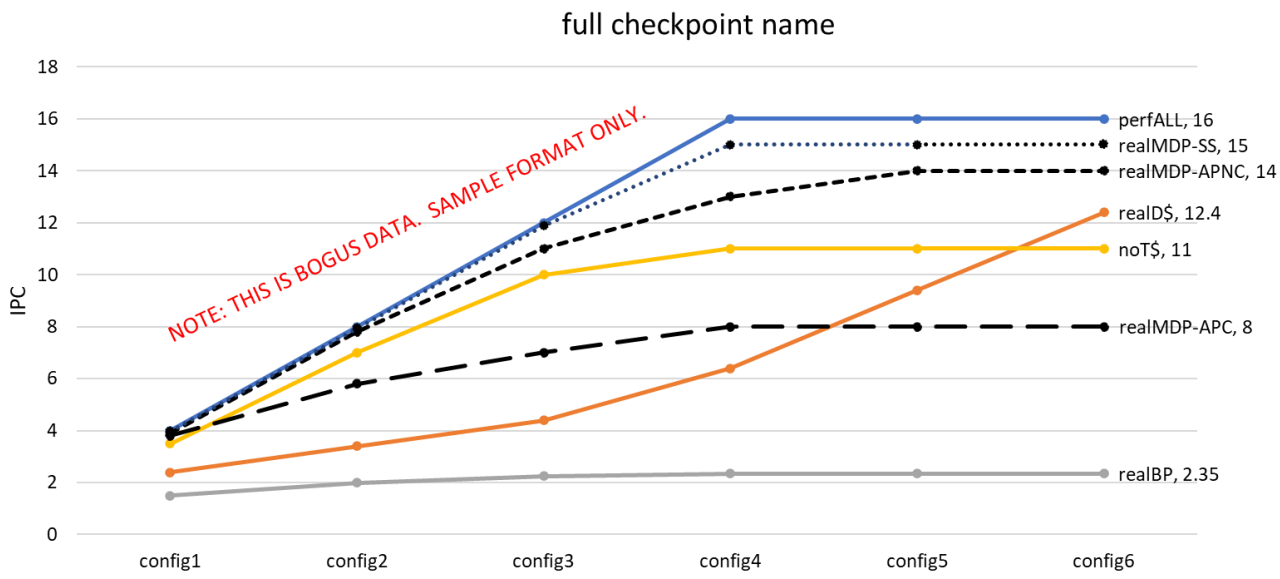
Read the document titled "ECE 721 Benchmarks and How to Run 721sim" that is posted on the Moodle site. It explains the benchmarks and how to set up run directories for launching 721sim jobs.

You will do experiments with the following benchmark checkpoints:

1. 401.bzip2_chicken_ref.1394.0.21.gz
2. 619.lbm_s_ref.16511.0.74.gz
3. 400.perlbench_splitmail_ref.2724.0.44.gz

3. Studies

For each benchmark checkpoint, produce a graph that looks like this:



The title of the graph is the full checkpoint name. The y-axis is IPC. The sizes of key ILP-extracting structures – Active List (AL), Load Queue (LQ)/Store Queue (SQ), and Issue Queue (IQ) – are varied on the x-axis across six different size configurations named config1, config2, config3, config4, config5, and config6. The six size configurations are shown in the table below. For example, config4 has 1024-entry AL, 512-entry LQ and 512-entry SQ, and 256-entry IQ. Each successive configuration doubles the sizes of the key structures. The table also shows the 721sim simulator flags for setting the sizes of the AL (`--al=[size]`), LQ/SQ (`--lsq=[size]`), and IQ (`--iq=[size]`).

name of size configuration	Active List (AL)	Load Queue (LQ)/ Store Queue (SQ)	Issue Queue (IQ)
config1	`--al=128`	`--lsq=64`	`--iq=32`

config2	`--al=256`	`--lsq=128`	`--iq=64`
config3	`--al=512`	`--lsq=256`	`--iq=128`
config4	`--al=1024`	`--lsq=512`	`--iq=256`
config5	`--al=2048`	`--lsq=1024`	`--iq=512`
config6	`--al=4096`	`--lsq=2048`	`--iq=1024`

The graph has seven curves with six datapoints each (due to varying config1 through config6 on the x-axis). The seven curves are explained in the following subsections.

3.1. perfALL (everything perfect)

The "perfALL" curve is an upper bound on performance (a limit study) due to having perfect branch prediction, perfect data and instruction caches, perfect trace cache, and oracle memory dependence prediction. The six datapoints can be generated using the following commands once you set up a run directory for a given benchmark checkpoint.

perfALL, config1:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=4,0 --perf=1,1,1,1 -t --al=128 --lsq=64 --iq=32'
```

perfALL, config2:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=4,0 --perf=1,1,1,1 -t --al=256 --lsq=128 --iq=64'
```

perfALL, config3:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=4,0 --perf=1,1,1,1 -t --al=512 --lsq=256 --iq=128'
```

perfALL, config4:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=4,0 --perf=1,1,1,1 -t --al=1024 --lsq=512 --iq=256'
```

perfALL, config5:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=4,0 --perf=1,1,1,1 -t --al=2048 --lsq=1024 --iq=512'
```

perfALL, config6:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=4,0 --perf=1,1,1,1 -t --al=4096 --lsq=2048 --iq=1024'
```

Notes:

- Branch prediction: perfect (`--perf=1,1,1,1`)
- L1 data cache: perfect (`--perf=1,1,1,1`)
- L1 instruction cache: perfect (`--perf=1,1,1,1`)
- Trace cache: enabled (`-t`) and perfect (`--perf=1,1,1,1`)
- Memory dependence prediction: oracle (`--mdp=4,0`)
- Miscellaneous:
  - `-L` (limit study) configures: fetch, dispatch, issue, and retire widths all 16; universal execution lanes with 1-cycle EX latency; ideal age-based issuing; 64 branch checkpoints (maximum supported by 721sim); FQ size of 64.
  - The `-e10000000` flag configures the simulator to simulate 10-million dynamic instructions (starting from the SimPoint, in our case, since we are using benchmark checkpoints).

3.2. realD$ (everything perfect except data cache)

The “realD$” curve is like “perfALL” except that perfect data cache is disabled to isolate the IPC impact of data cache misses.

realD$, config1:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=4,0 --perf=1,0,1,1 -t --al=128 --lsq=64 --iq=32'
```

realD$, config2:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=4,0 --perf=1,0,1,1 -t --al=256 --lsq=128 --iq=64'
```

realD$, config3:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=4,0 --perf=1,0,1,1 -t --al=512 --lsq=256 --iq=128'
```

realD$, config4:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=4,0 --perf=1,0,1,1 -t --al=1024 --lsq=512 --iq=256'
```

realD$, config5:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=4,0 --perf=1,0,1,1 -t --al=2048 --lsq=1024 --iq=512'
```

realD$, config6:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=4,0 --perf=1,0,1,1 -t --al=4096 --lsq=2048 --iq=1024'
```

3.3. realBP (everything perfect except branch prediction)

The “realBP” curve is like “perfALL” except that perfect branch prediction is disabled to isolate the IPC impact of branch mispredictions.

realBP, config1:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=4,0 --perf=0,1,1,1 -t --al=128 --lsq=64 --iq=32'
```

realBP, config2:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=4,0 --perf=0,1,1,1 -t --al=256 --lsq=128 --iq=64'
```

realBP, config3:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=4,0 --perf=0,1,1,1 -t --al=512 --lsq=256 --iq=128'
```

realBP, config4:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=4,0 --perf=0,1,1,1 -t --al=1024 --lsq=512 --iq=256'
```

realBP, config5:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=4,0 --perf=0,1,1,1 -t --al=2048 --lsq=1024 --iq=512'
```

realBP, config6:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=4,0 --perf=0,1,1,1 -t --al=4096 --lsq=2048 --iq=1024'
```

3.4. noT$ (everything perfect except there is no trace cache)

The “noT$” curve is like “perfALL” except that perfect trace cache is disabled (and there is no trace cache at all, not even a real one) to isolate the IPC impact of terminating fetch bundles at predicted-taken conditional branches and direct jumps (which otherwise do not terminate fetch bundles with a trace cache).

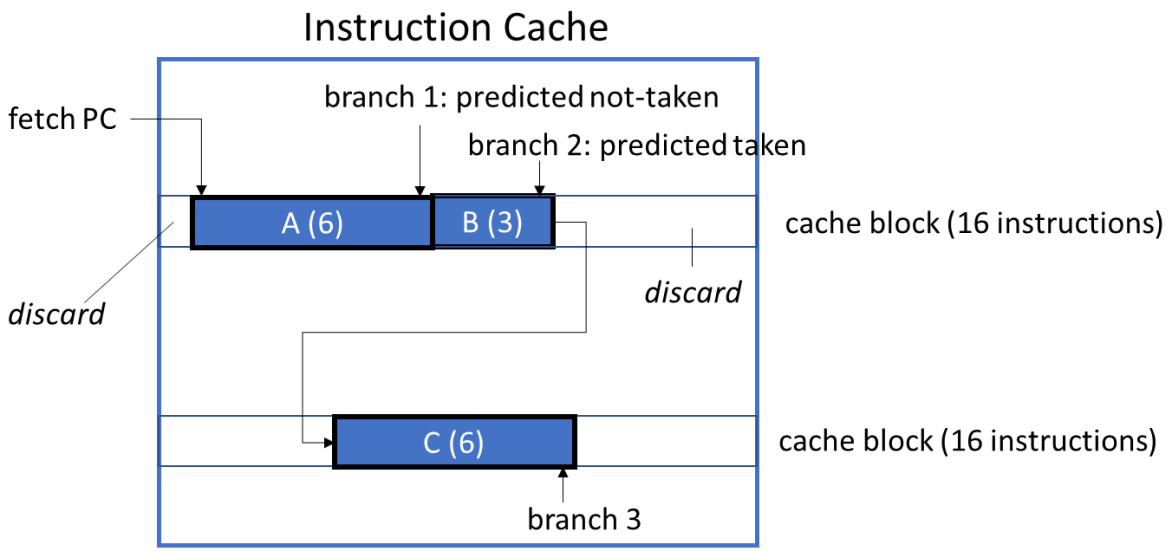
3.4.1. Background

Everything that is discussed in this section (and more) will be taught in upcoming lectures.

In this project, the fetch unit is configured with a multiple-branch predictor that can predict up to 3 conditional branches and 1 indirect branch each cycle.

For a processor with only a conventional instruction cache, the BTB locates all branches in the current instruction fetch bundle, steers conditional branch predictions to the conditional branches in the fetch bundle, and steers a predicted indirect target to the first indirect branch in the fetch bundle if there are any. The key point is that, with only a conventional instruction cache and BTB, the fetch bundle must be terminated at the first predicted-taken branch because the sequential instructions after this branch are not supposed to be fetched according to the branch prediction machinery: they are not part of the predicted path. The next fetch bundle, in the next fetch cycle, will start at the taken target of the predicted-taken branch. If predicted-taken branches are frequent, then the effective fetch width is significantly reduced as compared to the full fetch width of the processor, which in turn reduces IPC.

Consider the example in the figure below. From a hardware standpoint, a “basic block” is a sequential run of instructions that ends in a branch instruction. The figure shows three basic blocks, A, B, and C, that each end in a conditional branch. A, B, and C, contain 6 instructions, 3 instructions, and 6 instructions, respectively. Assume the current fetch PC points to the first instruction in basic block A. Assume the first two predictions supplied by the multiple-branch predictor are “1:not-taken, 2:taken” and the third prediction is not important for this discussion. Assume each cache block is 16 instructions long. With a conventional instruction cache, the fetch unit only fetches the first 16-instruction cache block containing A and B. The fetch unit takes this “raw” 16-instruction fetch bundle and extracts the desired “true” 9-instruction fetch bundle composed of just basic blocks A and B. This is due to the fact that we don’t want the sequential instructions prior to A (because the fetch PC points to the first instruction in A) and the branch prediction machinery is indicating to discard the sequential instructions after B, because its ending branch is predicted as taken. Thus, the effective fetch width in this first fetch cycle is 9 instructions, not 16 instructions. In the next fetch cycle, the processor will fetch another fetch bundle that starts with basic block C (and may or may not include additional basic blocks depending on where the first predicted-taken branch is in the next fetch bundle).



The key idea behind a trace cache is to enable fetching the non-sequential basic block C *in the same cycle* as basic blocks A and B. This means the effective fetch width in the first cycle would be 15 instructions instead of 9.

The table below shows criteria for terminating fetch bundles in this project, for perfALL (perfect trace cache) and noT$ (no trace cache at all). The crucial difference is that, with a perfect trace cache, predicted-taken conditional branches and direct jumps do not in and of themselves terminate fetch bundles.

	perfALL: fetch bundle is terminated at whichever of the following selected criteria comes first	noT$: fetch bundle is terminated at whichever of the following selected criteria comes first
16 instructions max	√	√
3 conditional branches max (stop at 3rd cond. branch) due to predictor throughput of 3/cycle	√	√
first predicted-taken conditional branch		√
jump direct		√
jump indirect	√	√
call direct	√	√
call indirect	√	√
return	√	√

3.4.2. noT$

NOTE: omit the `-t` flag so that trace cache is disabled.

noT$, config1:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=4,0 --perf=1,1,1,0 --al=128 --lsq=64 --iq=32'
```

noT$, config2:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=4,0 --perf=1,1,1,0 --al=256 --lsq=128 --iq=64'
```

noT$, config3:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=4,0 --perf=1,1,1,0 --al=512 --lsq=256 --iq=128'
```

noT$, config4:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=4,0 --perf=1,1,1,0 --al=1024 --lsq=512 --iq=256'
```

noT$, config5:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=4,0 --perf=1,1,1,0 --al=2048 --lsq=1024 --iq=512'
```

noT$, config6:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=4,0 --perf=1,1,1,0 --al=4096 --lsq=2048 --iq=1024'
```

3.5. realMDP-* (everything perfect except memory dependence prediction)

The "realMDP-*" curves are like "perfALL" except that real memory dependence prediction is used to isolate the IPC impact of imperfect memory dependence prediction.

3.5.1. Background

Everything that is discussed in this section (and more) will be taught in upcoming lectures. You should have had some introduction to the SQ/LQ in ECE 463/563.

A processor's memory dependence prediction policy applies when a load is ready to issue with respect to its source register dependencies, yet there are older stores still in the IQ. The load should wait in the IQ if it depends on one of these older stores, but we are missing three pieces of information *at the IQ* that are needed to make this determination:

1. The older stores' addresses are unknown because they haven't issued and executed yet.
2. The load's address is unknown until it issues and executes.
3. The load may depend on an older store that already issued, executed, and deposited its {address, value} in the SQ. Whether or not this is the case depends on the program order of all older issued and unissued stores, their addresses, and the load's address. We don't have any of this information *at the IQ*.

One policy is to predict that a given load doesn't depend on any older unissued stores. Let's refer to this policy as "always predict no conflict" (realMDP-APNC). With realMDP-APNC, a load issues without waiting for older unissued stores. This policy is optimal when the load, in fact, doesn't depend on any older unissued stores. But if it does depend on one of the older unissued stores, then the load issued prematurely and got a stale value from the D$/SQ. Later, when its producer store issues, executes, and searches the LQ, the load will get marked as mispredicted in the Active List. When the mispredicted load reaches the head of the Active List, it and all subsequent instructions are squashed and instruction fetching resumes from the mispredicted load (it will execute correctly the second time around because all prior stores have committed). We call this mispredicted load a "load violation". Policy realMDP-APNC has the potential to cause many load violations. On the other hand, it doesn't introduce any false dependencies.

At the other extreme, we could delay issuing a given load until all older unissued stores have issued. This policy essentially predicts that the load depends on one of the older unissued stores, but makes no attempt to predict which one, and thus waits for all of them to issue. Let's refer to this policy as "always predict conflict" (realMDP-APC). Policy realMDP-APC has the potential to delay loads in the IQ much longer than necessary. On the other hand, it eliminates load violations.

Modern processors use some kind of memory dependence predictor that is referenced by all stores and loads, prior to their dispatch into the IQ, to predict specific store-load and store-store pairs that should issue serially from the IQ. Each IQ entry is augmented with another source tag to enable a store instruction to wake-up other stores and loads when it issues, and in this way serialize the issuing of specific stores and loads that are predicted to have the same address. For example, the memory dependence predictor may cause two stores and a load to issue serially if it believes they are to the same address (in this example, store1 is oldest, store2 is next-oldest, and the load is youngest): store1 → store2 → load. If, in fact, they all have the same address, then their serialization ensures that

when the load searches the SQ, it observes an address match with both store1 and store2 and correctly gets store2's value via the SQ program-ordered search. Using an accurate memory dependence predictor (1) significantly reduces the number of load violations as compared to realMDP-APNC, and (2) significantly streamlines the issuing of loads from the IQ as compared to realMDP-APC. 721sim can be configured to use the "Store Sets" memory dependence predictor, which we will learn about via a required paper and lectures later in the semester. Let's refer to it as realMDP-SS.

721sim's oracle memory dependence prediction mode leverages *oracle* knowledge of store and load addresses (prior to the stores and loads issuing) to serially issue only store-store and store-load pairs that truly conflict. It eliminates both load violations and unnecessary serializations. PerfALL uses oracle memory dependence prediction.

3.5.2. realMDP-APC: Real memory dependence prediction, always predict conflict

realMDP-APC, config1:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=6,0 --perf=1,1,1,1 -t --al=128 --lsq=64 --iq=32'
```

realMDP-APC, config2:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=6,0 --perf=1,1,1,1 -t --al=256 --lsq=128 --iq=64'
```

realMDP-APC, config3:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=6,0 --perf=1,1,1,1 -t --al=512 --lsq=256 --iq=128'
```

realMDP-APC, config4:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=6,0 --perf=1,1,1,1 -t --al=1024 --lsq=512 --iq=256'
```

realMDP-APC, config5:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=6,0 --perf=1,1,1,1 -t --al=2048 --lsq=1024 --iq=512'
```

realMDP-APC, config6:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=6,0 --perf=1,1,1,1 -t --al=4096 --lsq=2048 --iq=1024'
```

3.5.3. realMDP-APNC: Real memory dependence prediction, always predict no-conflict

realMDP-APNC, config1:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=1,0 --perf=1,1,1,1 -t --al=128 --lsq=64 --iq=32'
```

realMDP-APNC, config2:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=1,0 --perf=1,1,1,1 -t --al=256 --lsq=128 --iq=64'
```

realMDP-APNC, config3:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=1,0 --perf=1,1,1,1 -t --al=512 --lsq=256 --iq=128'
```

realMDP-APNC, config4:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=1,0 --perf=1,1,1,1 -t --al=1024 --lsq=512 --iq=256'
```

realMDP-APNC, config5:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=1,0 --perf=1,1,1,1 -t --al=2048 --lsq=1024 --iq=512'
```

realMDP-APNC, config6:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=1,0 --perf=1,1,1,1 -t --al=4096 --lsq=2048 --iq=1024'
```

3.5.4. realMDP-SS: Real memory dependence prediction, Store Sets

realMDP-SS, config1:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=5,0 --perf=1,1,1,1 -t --al=128 --lsq=64 --iq=32'
```

realMDP-SS, config2:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=5,0 --perf=1,1,1,1 -t --al=256 --lsq=128 --iq=64'
```

realMDP-SS, config3:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=5,0 --perf=1,1,1,1 -t --al=512 --lsq=256 --iq=128'
```

realMDP-SS, config4:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=5,0 --perf=1,1,1,1 -t --al=1024 --lsq=512 --iq=256'
```

realMDP-SS, config5:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=5,0 --perf=1,1,1,1 -t --al=2048 --lsq=1024 --iq=512'
```

realMDP-SS, config6:

```
make cleanrun SIM_FLAGS_EXTRA='-e10000000 -L --mdp=5,0 --perf=1,1,1,1 -t --al=4096 --lsq=2048 --iq=1024'
```

4. What to hand in

You will submit your results and answer questions about them via a quiz posted on the Moodle site.

5. Late policy

As stated in the syllabus:

Late assignment policy:

- Quizzes: Because of the way quizzes are administered and graded, missing the deadline means your quiz is not submitted at all (0 for the quiz). (Specifically, Moodle does not provide an automated method for applying a late penalty, so the TA would have to check timeliness of each student's quiz and manually adjust scores accordingly, negating the huge benefits of auto-grading.) Please also see the policy on excused/unexcused absences in this syllabus.
- Project 1: Project 1 is submitted as a Moodle quiz. Therefore, the same policy applies to Project 1 as other quizzes: failing to make the Project 1 quiz attempt (*i.e.*, not entering your ILP limit study results and analysis) will result in a 0 grade for Project 1, even if you did the work for Project 1.